

Computergrafik

PD Prof. Dr. rer nat.
Marco Block-Berlitz

Sommersemester 2026



Refraktion, Schatten, Reflexion und Kaustik

„Das Wasser nimmt jede Form an, in die man es gießt, und doch bleibt es sich selbst treu.“ (sinngemäß nach Katsushika Hokusai)

PD Prof. Dr. Marco Block-Berlitz



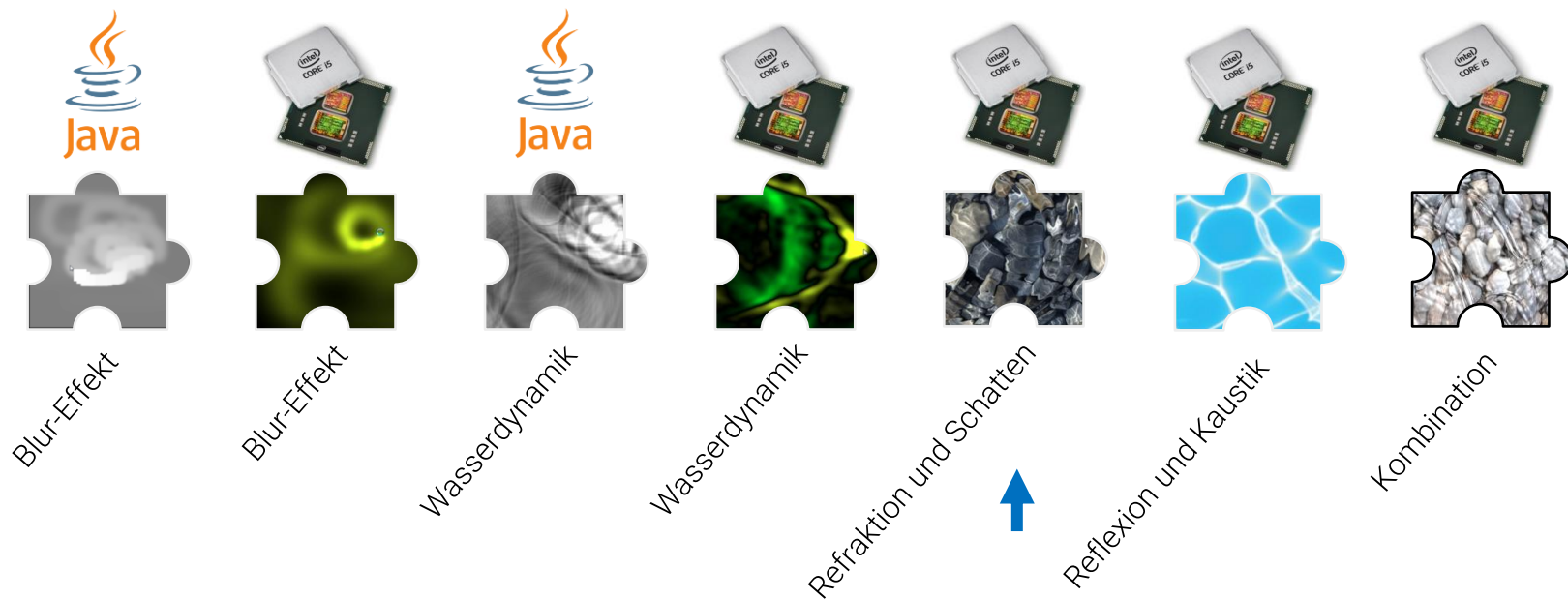
Ziele und Inhalt

- Projektstufe 5 (Refraktion und Schatten)
- Was ist Refraktion
- Bestimmung der Normalen der Wasseroberfläche
- Bestimmung der Refraktion
- Gesetz von Snell
- Shaderstrategie für Refraktion
- Schatteneffekt durch Gradienten
- Projektstufe 6 (Reflexion und Kaustik)
- Was ist Reflexion
- Umsetzungs idee im Shader
- Bestimmung des Reflexionsvektors
- Shader-Code der Reflexion
- Der Kaustik-Effekt
- Umsetzungs idee des Kaustik-Effektes
- Shader-Code des Kaustik-Effektes
- Fragestellungen zum Vorlesungsteil
- Praktische Aufgaben



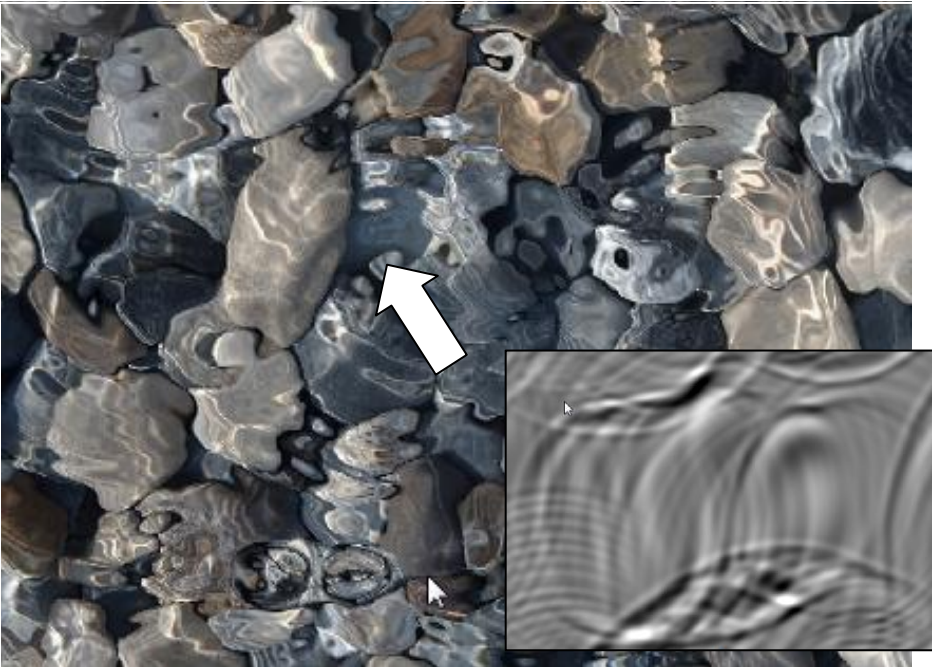
Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.



Projektziel – Stufe 5

In der fünften Projektstufe wollen wir die Wasserdynamik nutzen, um die **Refraktion** an der Wasseroberfläche zu ermitteln. Jetzt können wir den typischen Versatz der Bodentextur bei einem gegebenen Wellenmuster darstellen. Nebenbei erhalten wir einen **nützlichen Schatteneffekt**, den wir ebenfalls nutzen können.



Projektstufe 5

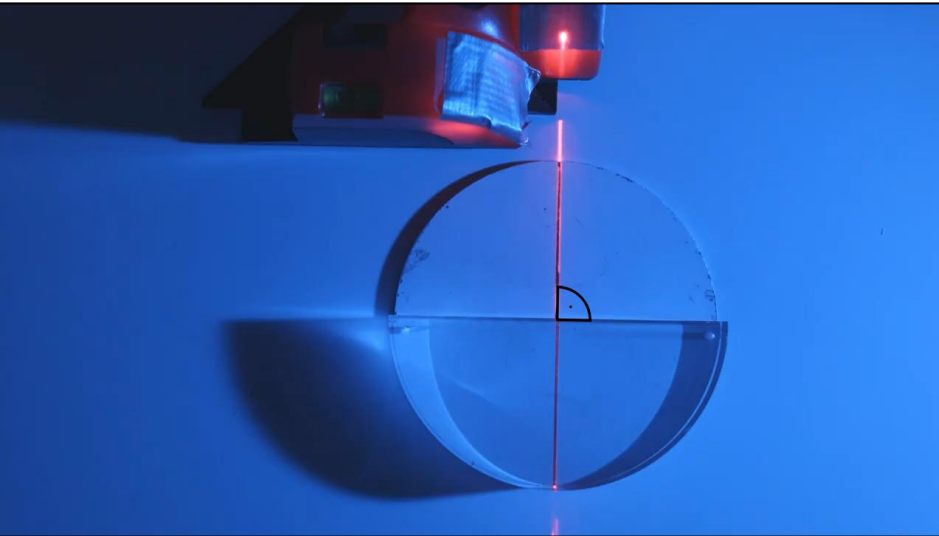
- der Hintergrundbereich soll dem Untergrund eines Baches entsprechen
- wir wollen die Wellendynamik für die Refraktion einsetzen
- ein kleiner netter Schatteneffekt soll hinzugefügt werden



Was bedeutet eigentlich **Refraktion**?

Refraktion bedeutet Lichtbrechung

Unter der **Refraktion** verstehen wir die Lichtbrechung, die entsteht, wenn das Licht das Medium wechselt. Typischerweise kennt man den Effekt bei der Brechung eines Lichtstrahls beim Übergang von Luft zu Wasser. Der Effekt kann durch einen Laser gut sichtbar gemacht werden [1].



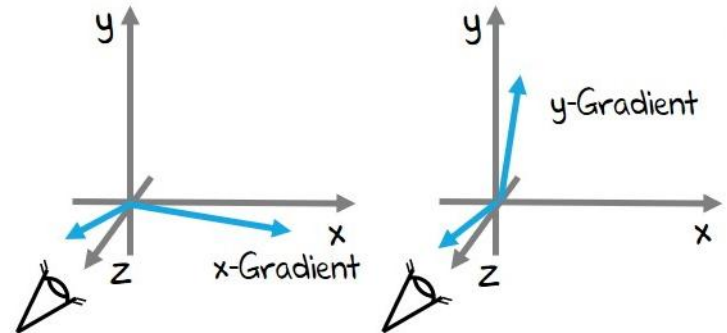
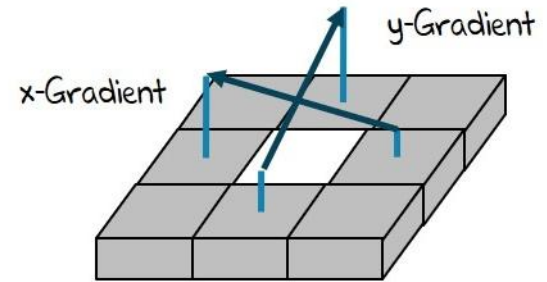
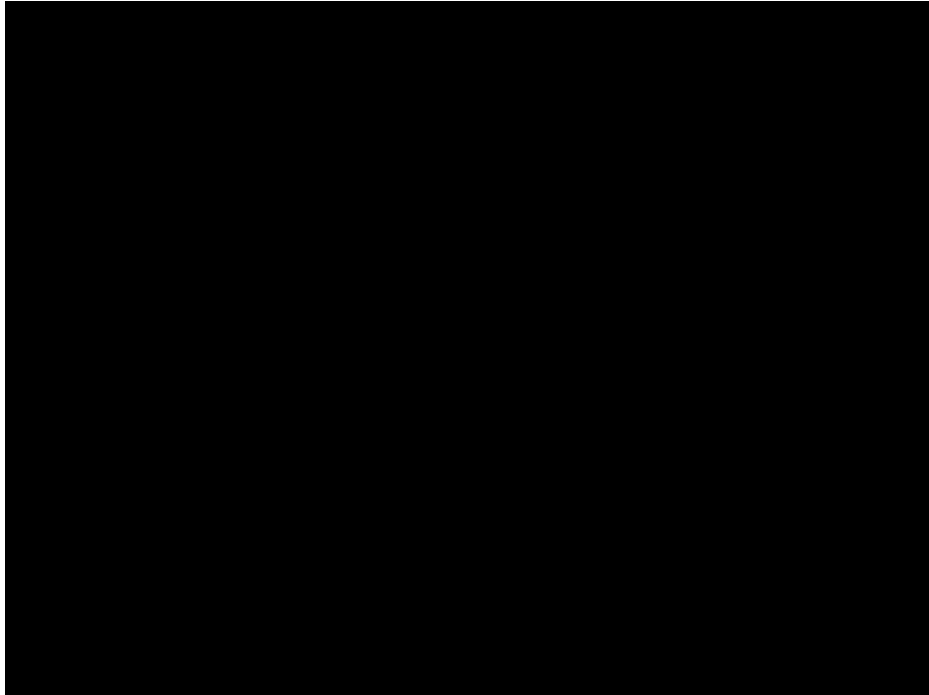
[1] <https://www.youtube.com/watch?v=fTtjKozHdIU>



Wie können wir die **Normale** an der Wasseroberfläche bestimmen?

Normale der Wasseroberfläche bestimmen

Wir erinnern uns an das Ergebnis der [Wasserdynamik](#) (Projektstufe 4). In **Framebuffer1** haben wir an jedem (x,y)-Punkt einen **Float**-Wert gespeichert, der der Höhe der Wasseroberfläche an dieser Stelle entspricht.



Normale der Wasseroberfläche bestimmen

Zunächst ermitteln wir die beiden Steigungen ([Gradienten](#)) in x - und y -Richtung durch die Differenz der vorliegenden Nachbarschaftswerte:

$$g_x(x, y) = w(x - 1, y) - w(x + 1, y)$$

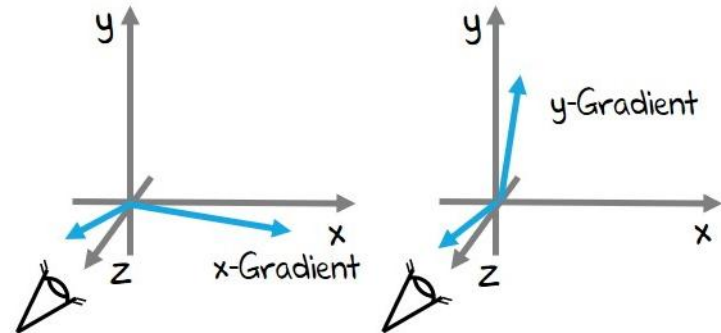
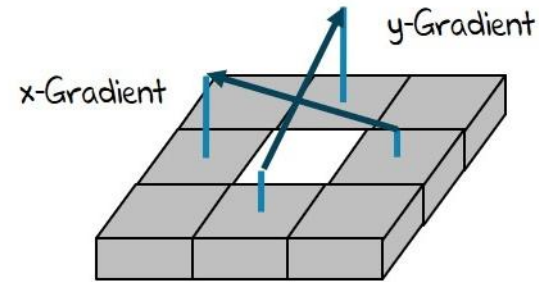
$$g_y(x, y) = w(x, y - 1) - w(x, y + 1)$$

Jetzt aus den Differenzen noch Vektoren konstruieren:

$$\vec{g}_x = (1, 0, g_x) \text{ und } \vec{g}_y = (0, 1, g_y)$$

Die [Normale](#) ergibt sich jetzt aus dem Kreuzprodukt beider Gradienten:

$$\vec{n} = \vec{g}_x \times \vec{g}_y$$



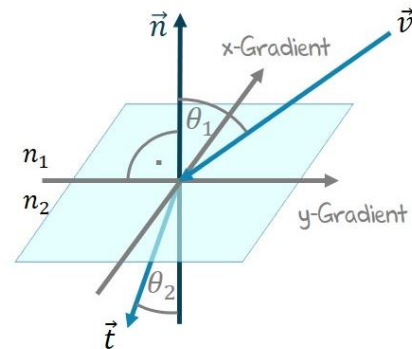


Wie berechnen wir jetzt die Refraktion?

Refraktionsvektor bestimmen

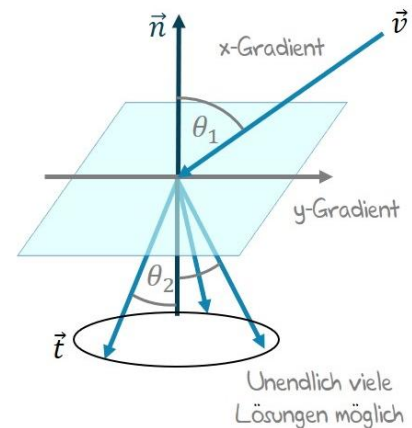
Der Sichtstrahl $\vec{v}$ des Betrachters, beginnend im Medium Luft n_1 , soll für jede Wellenposition beim Eintritt in das Medium Wasser n_2 korrekt umgelenkt werden. Glücklicherweise ist der Übergang zwischen zwei Medien ein bekanntes Problem mit einer einfachen Lösung.

Für die Berechnung verwenden wir das [Gesetz von Snell](#): $\frac{\sin(\theta_1)}{\sin(\theta_2)} = \frac{n_2}{n_1}$



Jetzt sind wir in der Lage, die Normale an jeder Position der Wasseroberfläche anzugeben. Um den [Refraktionsvektor](#) $\vec{t}$ zu bestimmen, genügt die Information von Snell allein allerdings nicht aus, da es unendlich viele Möglichkeiten für den Lösungsvektor bei gleichem Winkel θ_2 gibt.

$$\text{Die Lösung lautet: } \vec{t} = \frac{n_1}{n_2} \vec{v} + \left(\frac{n_1}{n_2} \cos(\theta_1) - \sqrt{1 - \sin^2(\theta_2)} \right) \vec{n}$$



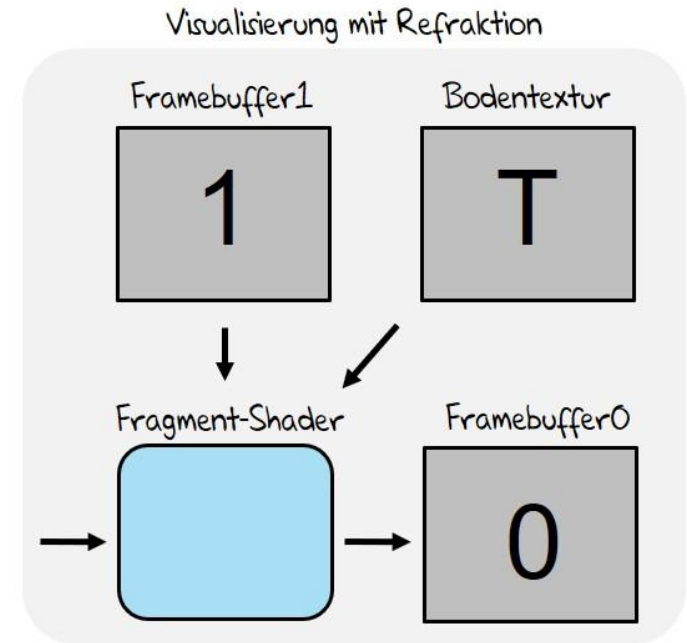
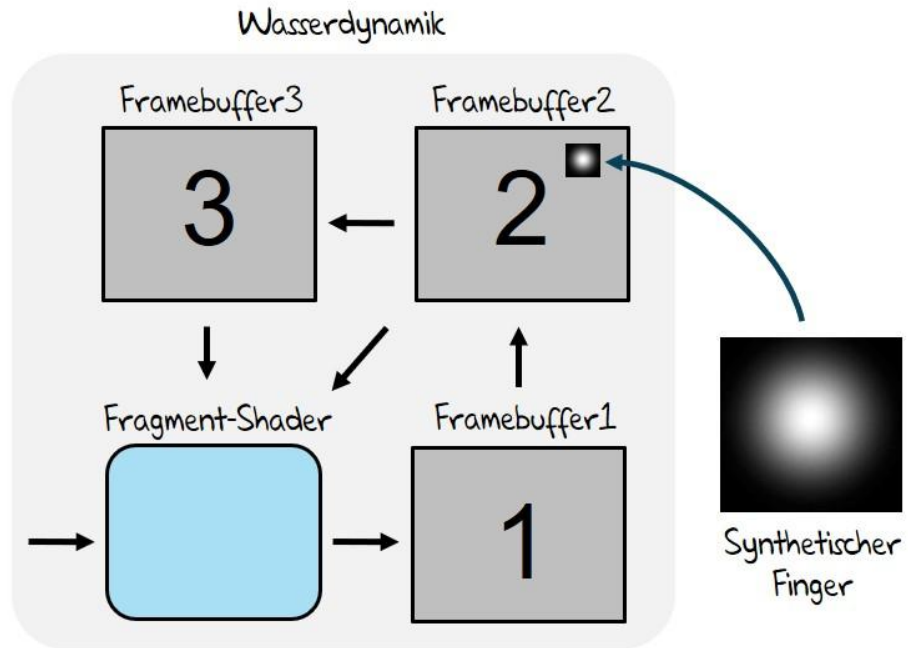
Glücklicherweise steht diese Funktion bereits in GLSL mit **refract** zur Verfügung.



Wir beginnen wie so oft mit der Shaderstrategie

Shader-Strategie für die Refraktion

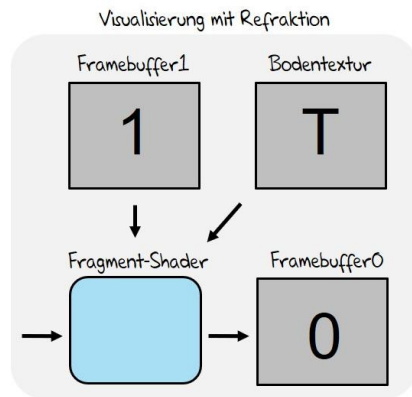
Wir benötigen drei Framebuffer für die Realisierung der Wasserdynamik im Shader. Mit der Hinzunahme eines synthetischen Fingers gewinnt die Darstellung der Wellendynamik deutlich an Qualität. Um das Ergebnis der **Refraktion** sichtbar zu machen, müssen wir für die Visualisierung eine **Bodentextur** hinzunehmen.



Implementierung Fragment-Shader Refraktion

Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Visualisierung an:

```
uniform sampler2D tex1;  
uniform sampler2D texT;  
uniform vec2 s;  
uniform float scale;  
  
void main (void) {  
    float g_x =(texture2D(tex1, gl_TexCoord[0].st + vec2(-s.x, 0)) -  
                texture2D(tex1, gl_TexCoord[0].st + vec2( s.x, 0))).x;  
    float g_y =(texture2D(tex1, gl_TexCoord[0].st + vec2(0, -s.y)) -  
                texture2D(tex1, gl_TexCoord[0].st + vec2(0,  s.y))).y;  
    vec3 n_surface = normalize(cross(vec3(1.0, 0.0, g_x), vec3(0.0, 1.0, g_y)));  
    vec3 n_view     = vec3(0.0, 0.0, 1.0);  
    vec3 refractV   = refract(n_view, n_surface, 1.33);  
    gl_FragColor    = texture2D(texT, gl_TexCoord[0].st + refractV.xy*s*scale);  
}
```

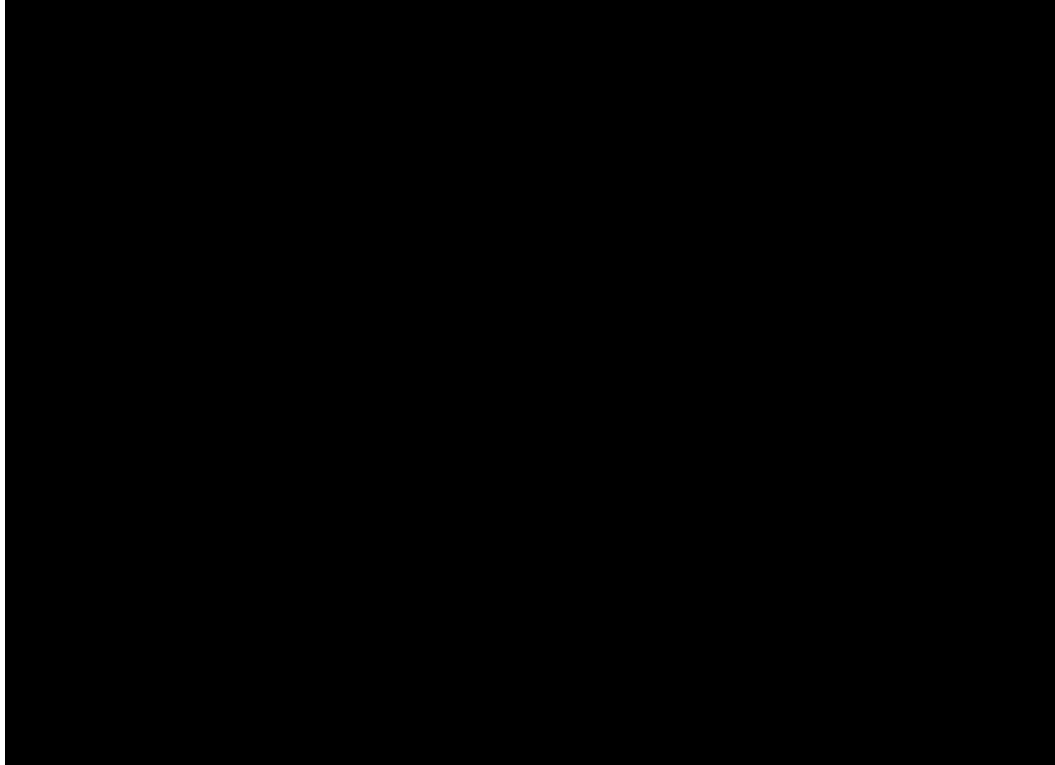




Schauen wir uns das Ergebnis an.

Implementierung im Shader

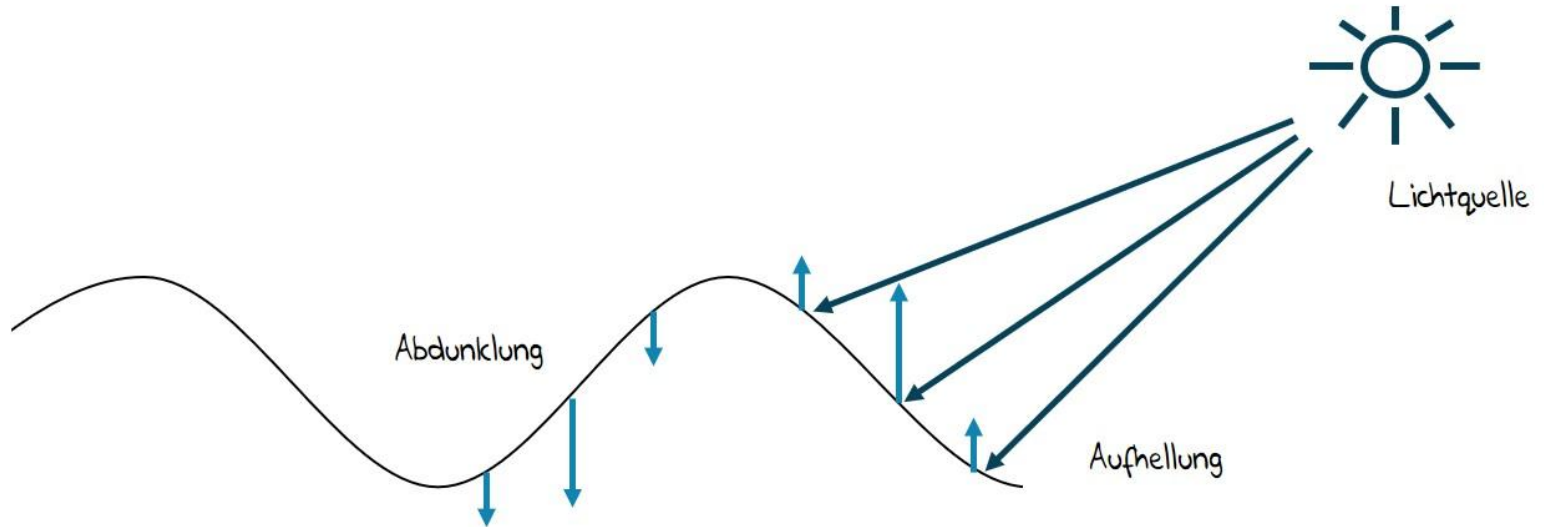
Schauen wir uns das Ergebnis der Kombination aus Wellendynamik und Refraktion an:



Programmcode:
kapitel02/RefraktionZurBodenTextur.java

Schatteneffekt durch Gradienten

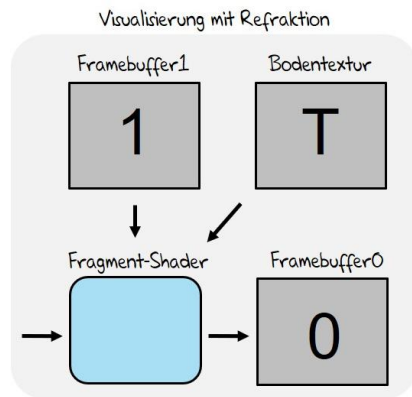
Einfacher [Schatteneffekt](#) durch Gradienten aus x - und y -Richtung. Die Gradienten werden aufsummiert und liefern bei positivem Vorzeichen eine Aufhellung und bei negativem Vorzeichen eine Verdunklung des entsprechenden Pixels.



Implementierung Fragment-Shader Schatten

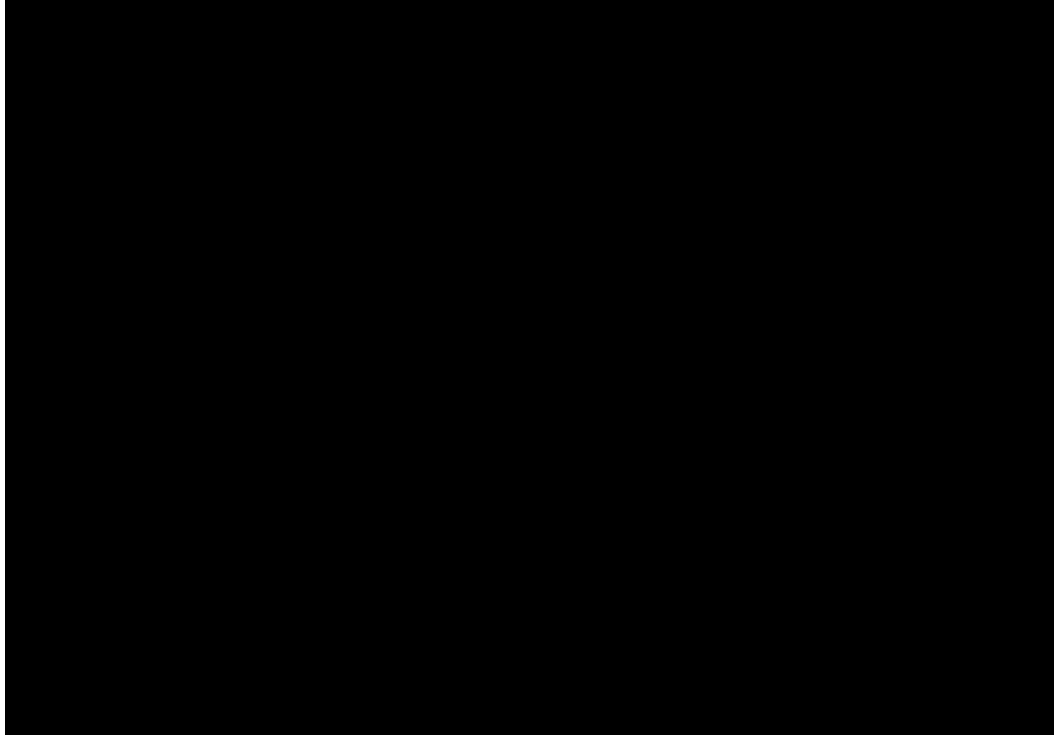
Schauen wir uns jetzt den kompletten Fragment-Shadercode für die Visualisierung an:

```
uniform sampler2D tex1;  
uniform sampler2D texT;  
uniform vec2 s;  
uniform float scale;  
  
void main (void) {  
    float g_x =(texture2D(tex1, gl_TexCoord[0].st + vec2(-s.x, 0)) -  
                texture2D(tex1, gl_TexCoord[0].st + vec2( s.x, 0))).x;  
    float g_y =(texture2D(tex1, gl_TexCoord[0].st + vec2(0, -s.y)) -  
                texture2D(tex1, gl_TexCoord[0].st + vec2(0,  s.y))).y;  
    float shadow    = 1.0 + 5.0 * (g_x - g_y);  
    vec4 shadowCol  = vec4(shadow, shadow, shadow, 1.0);  
    gl_FragColor    = shadowCol;  
}
```



Implementierung im Shader

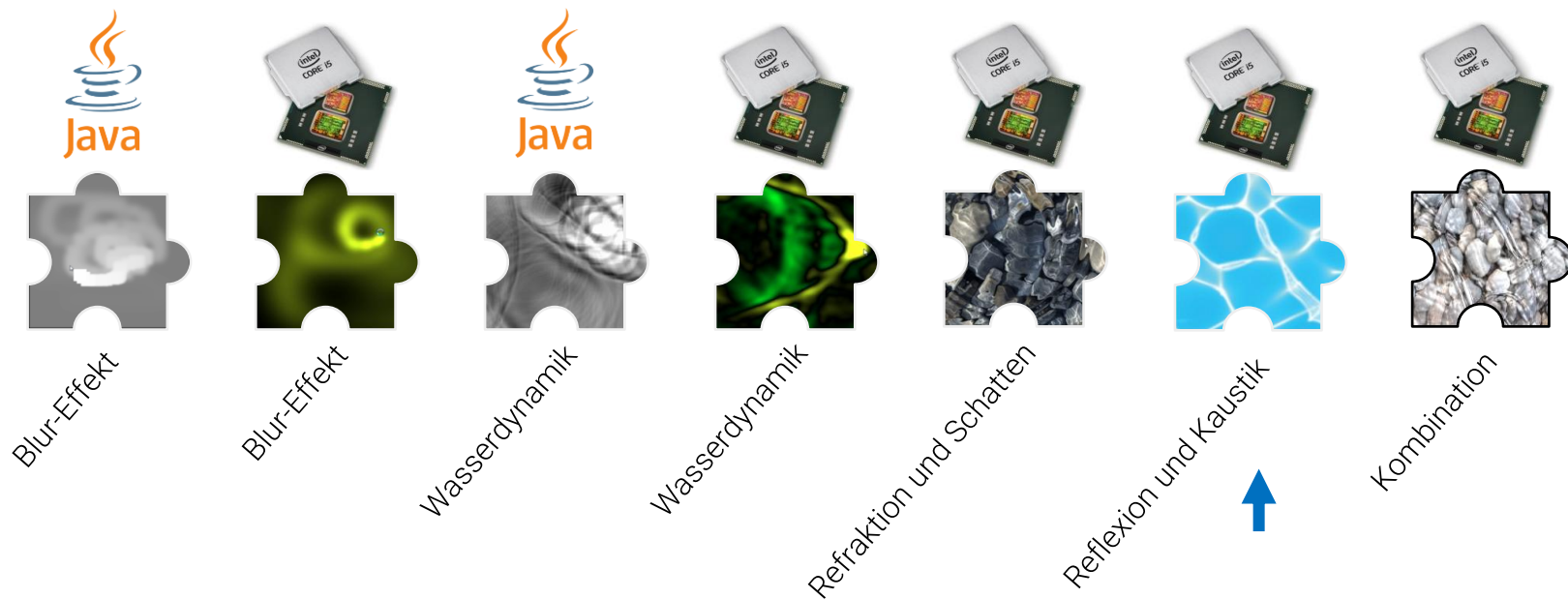
Schauen wir uns das Ergebnis des einfachen Schatteneffekts an:



Programmcode:
kapitel02/SchattenEffekt.java

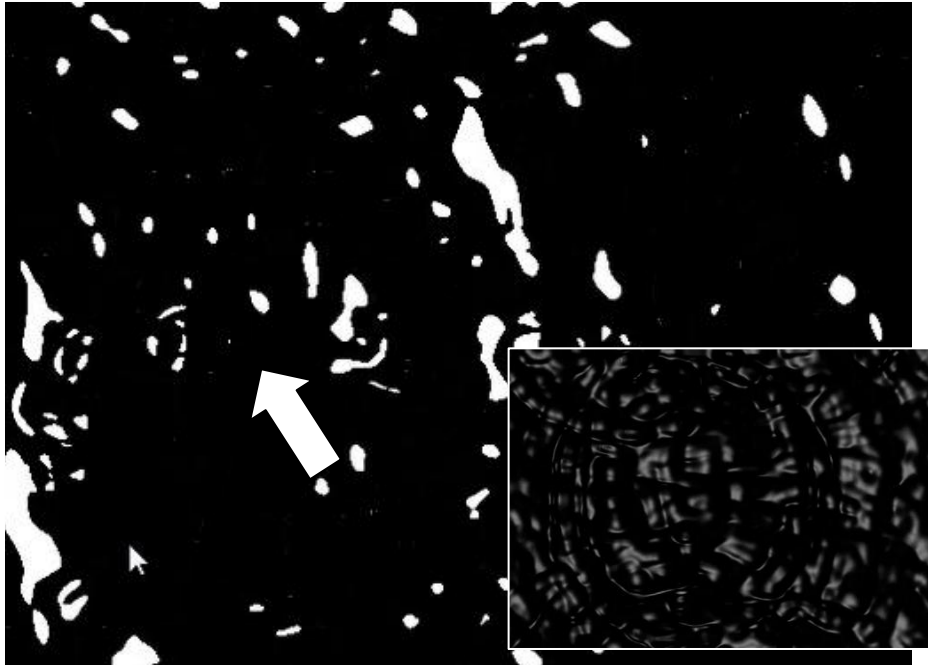
Interaktives Wasser – Projektetappen

Wasser hat in diesem Kontext schon immer eine besondere Anziehungskraft ausgeübt, symbolisiert es doch den Ursprung und das Grundbedürfnis des Lebens. Das folgende Projekt widmet sich dem Motto „Es war einmal das Wasser ...“ und wird Schritt für Schritt in die [Shaderprogrammierung mit GLSL](#) einführen und so – ganz nebenbei – die notwendigen mathematischen und physikalischen Konzepte behandeln. Beiläufig sehen wir instruktive Shaderbeispiele und machen uns mit GLSL vertraut.



Projektziel – Stufe 6

In der sechsten Projektstufe wollen wir auf der Wasseroberfläche Reflexionsanteile einer Lichtquelle hinzufügen. Diese Lichtquelle hat auch Einfluss auf die Beleuchtung des Gewässergrundes durch den sogenannten Kaustik-Effekt.



Projektstufe 6

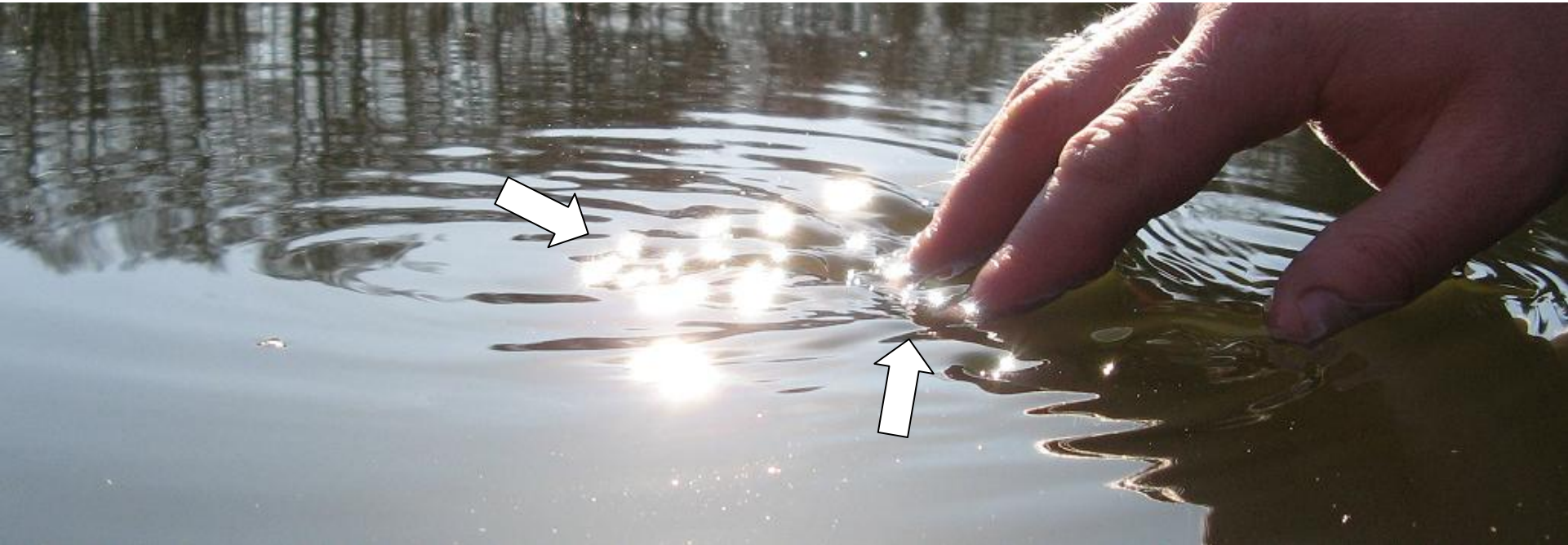
- zusätzliche Reflexionsanteile einer Lichtquelle am Wellenkamm (Mond)
- Bündelung und Aufweitung des Lichts am Untergrund simulieren (Kaustik-Effekt)



Was bedeutet Reflexion?

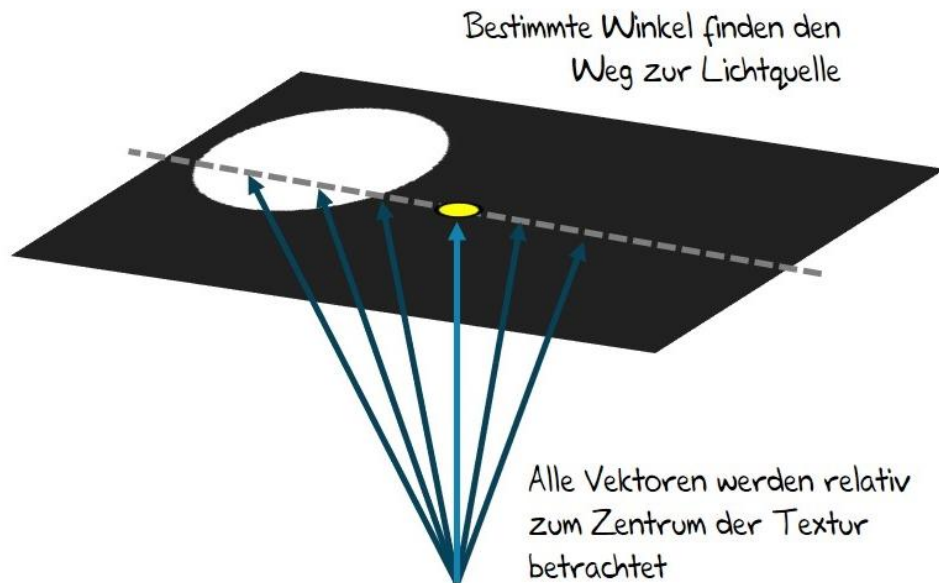
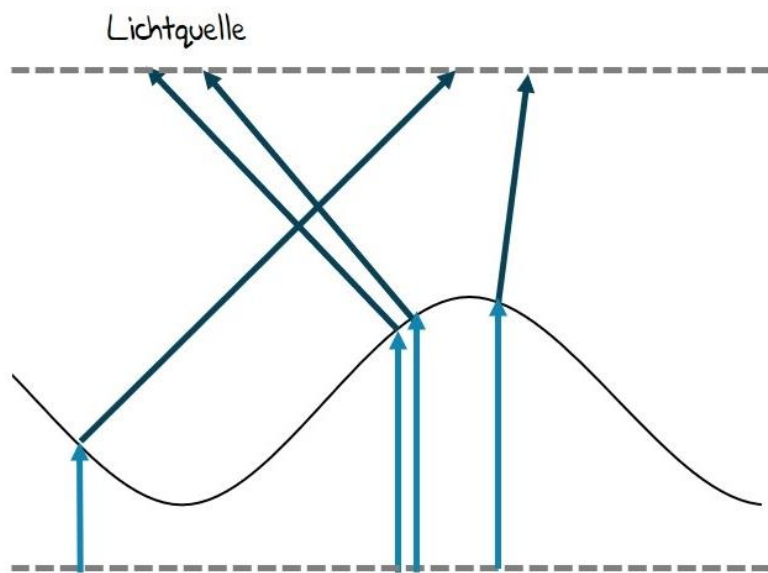
Reflexionen an der Wasseroberfläche

Mit der [Reflexion](#) haben wir einen sehr interessanten Effekt zu besprechen, von dem die Visualisierung der Wassersimulation enorm profitieren wird. Zusätzliche Reflexionsanteile einer Lichtquelle (Mondeffekt) sollen am Wellenkamm gezeigt werden.



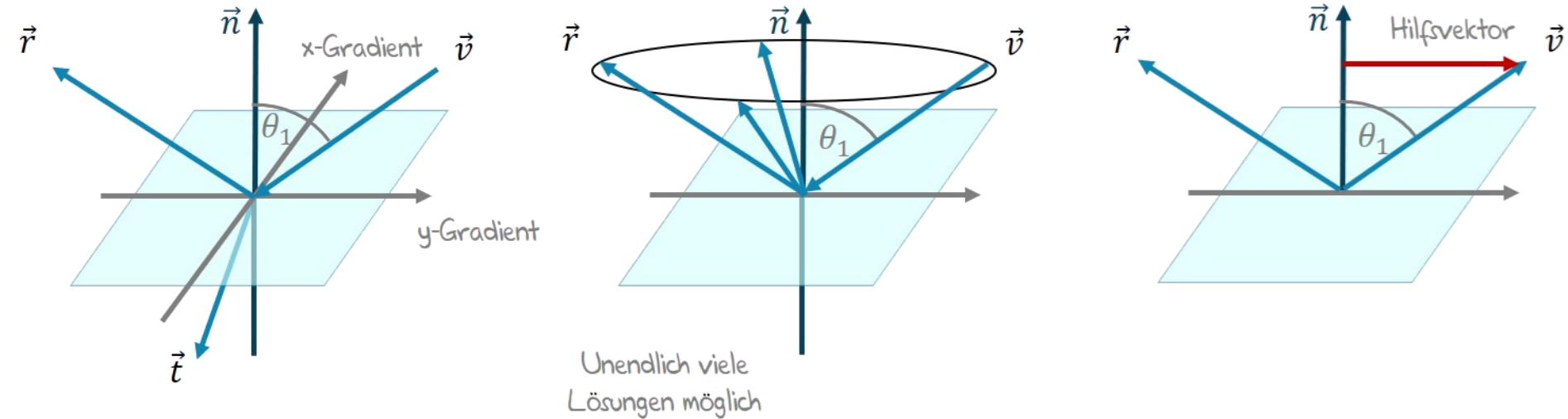
Umsetzungsidee der Reflexion

Die Reflexionsvektoren (dunkelblau) werden wir auf eine Textur mit einer Lichtquelle schicken. Alle Vektoren werden zunächst relativ zur Textur zentriert und auf diese Weise nur von einer Position aus betrachtet. Die Vektoren, die jetzt in eine bestimmte Richtung (also direkt zu einer Lichtquelle) zeigen, wollen wir heller darstellen. Dazu nutzen wir eine einfache Textur, auf der ein **Mond** abgebildet ist. Das führt später bei der Visualisierung zu gut umrandeten, spiegelnden Lichtelementen.



Reflexionsvektor direkt bestimmen

Den Refraktionsvektor $\vec{t}$ haben wir bereits angegeben. Die Berechnung des Reflexionsvektors $\vec{r}$ ist viel einfacher zu konstruieren. Links: Einfallswinkel gleich Ausfallswinkel. Mitte: Der Winkel allein genügt nicht, da unendlich viele Lösungen möglich sind. Rechts: In Wahrheit schaut $\vec{v}$ in Richtung des Betrachters. Wir hatten die Pfeile bisher andersherum dargestellt, damit die Sichtichtung besser verständlich ist. Durch einen Hilfsvektor können wir $\vec{r}$ direkt bestimmen.



Reflexionsvektor direkt bestimmen

Den **Reflexionsvektor** $\vec{r}$ können wir leicht berechnen:

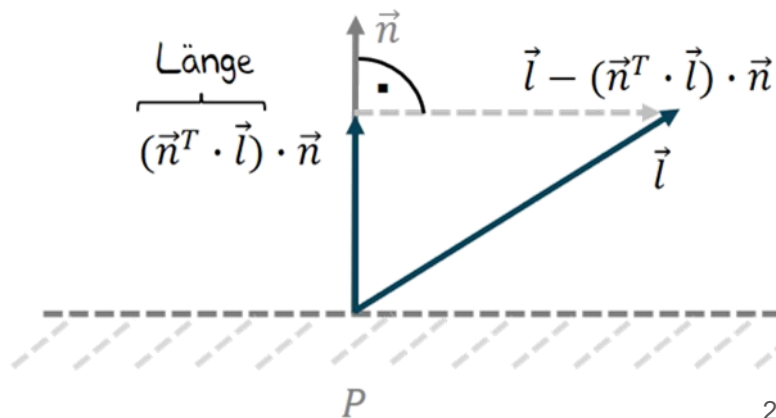
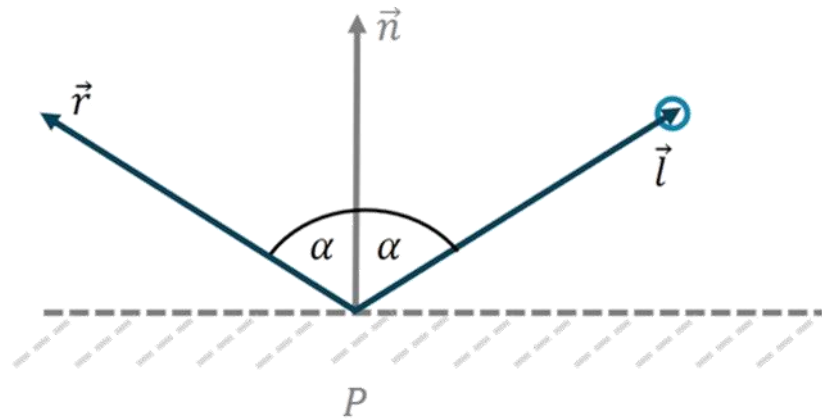
$$\begin{aligned}\vec{r} &= \vec{l} - 2 \cdot [\vec{l} - (\vec{n}^t \vec{l}) \cdot \vec{n}] \\ &= \vec{l} - 2 \cdot \vec{l} + 2 \cdot (\vec{n}^t \vec{l}) \cdot \vec{n} \\ &= -\vec{l} + 2 \cdot (\vec{n}^t \vec{l}) \cdot \vec{n} \\ &= 2 \cdot (\vec{n}^t \vec{l}) \cdot \vec{n} - \vec{l}\end{aligned}$$

Der Reflexionsvektor $\vec{r}$ sollte für weitere Berechnungen noch **normiert** werden:

$$\vec{r} = \text{norm}(2 \cdot (\vec{n}^t \vec{l}) \cdot \vec{n} - \vec{l})$$

Glücklicherweise steht diese Funktion bereits in GLSL mit **reflect** zur Verfügung.

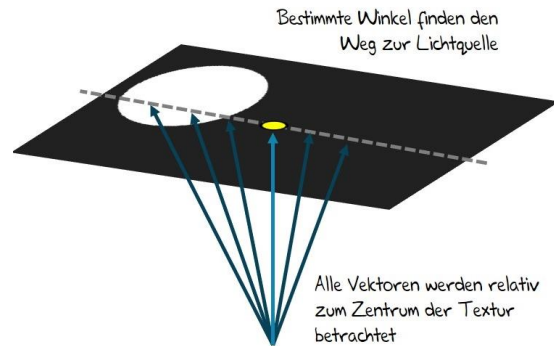
Wir sehen hier schon, dass es etwas aufwändig ist, den Reflexionsvektor für alle Pixel zu bestimmen. Später in Beleuchtung und Rendering lernen wir noch Tricks zur Optimierung kennen.



Implementierung Fragment-Shader Reflexion

Schauen wir uns jetzt den Fragment-Shadercode für die [Reflexion](#) an:

```
...  
uniform sampler2D tex3; // mond  
  
void main (void) {  
    ...  
    vec4 mondRefl = texture2D(tex3, 5.0*reflectVector.xy + vec2(0.5, 0.5));  
    ...  
}
```



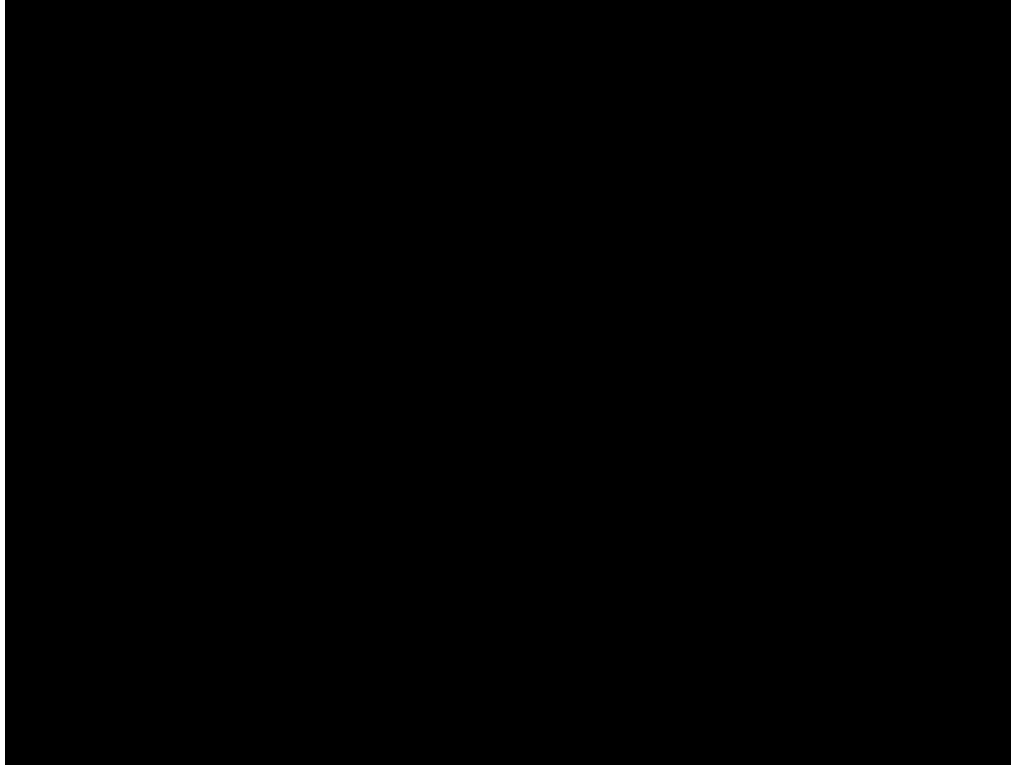
Reflexions-Effekt isoliert betrachtet

Hier sehen wir das isolierte Ergebnis der [Reflexion](#) ohne die bereits bekannten Effekte, wie Refraktion und Schatten.



Implementierung im Shader

Schauen wir uns das Ergebnis der Reflexion an:



Programmcode:
kapitel02/MondReflexion.java



Kommen wir jetzt zum **Kaustik-Effekt**.

Kaustik unter Wasser



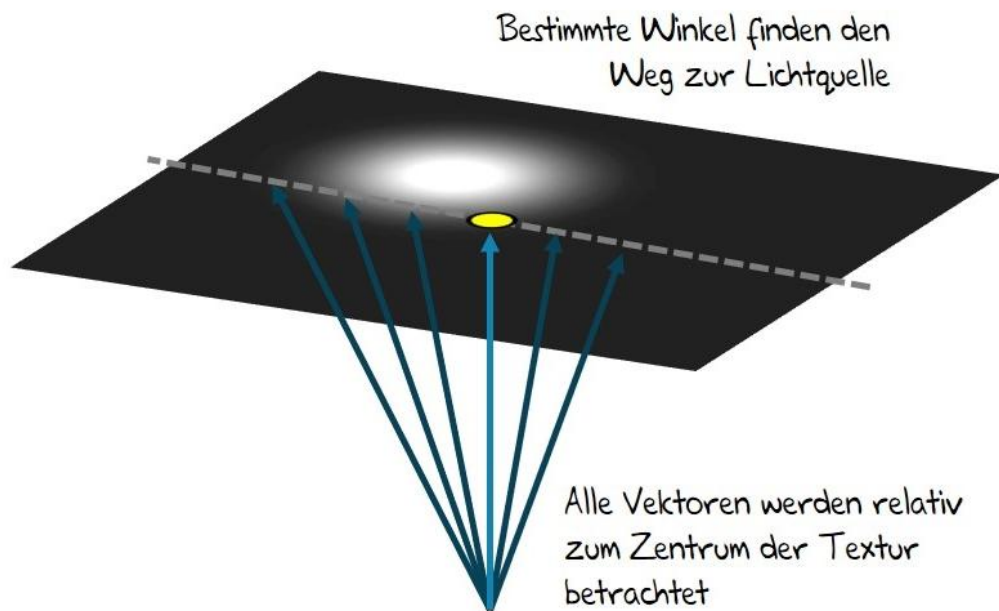
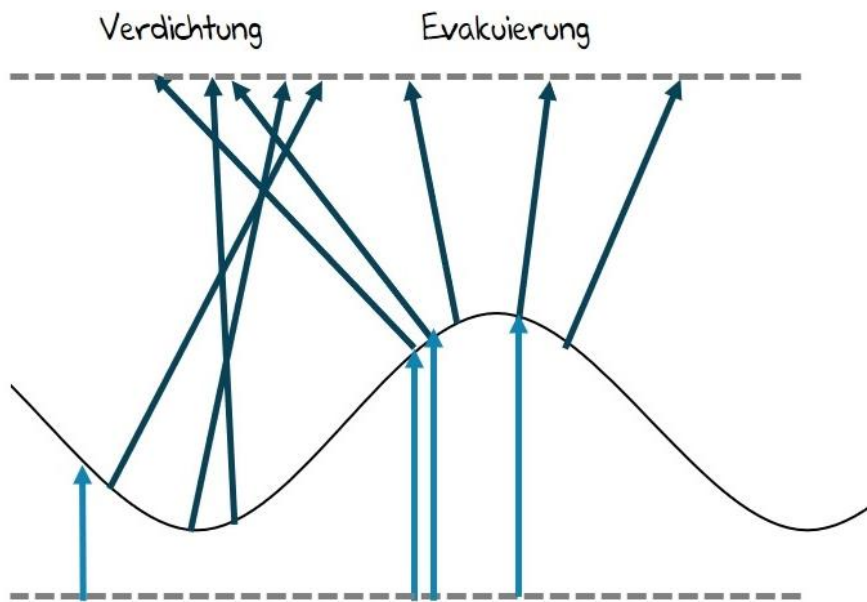
[1] <https://youtu.be/M2yzM4VwQeU?si=hrKmZvRrFsXfagVY>

[2] <https://youtu.be/qwz88S1P0os?si=HWocX6LookUR9VJI>

[3] <https://youtu.be/nK3oiBb9HEA?si=ho3R4c2hxPdkl4nH>

Umsetzungsidee des Kaustik-Effekts

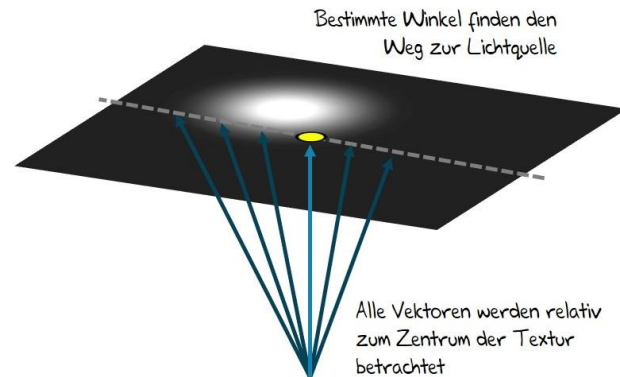
Beim **Kaustikeffekt** sind Aufweitungen und Bündelungen der Lichtstrahlen zu beobachten. Analog zum Reflexionseffekt wollen wir die abgelenkten Lichtstrahlen in Bezug zu den Winkeln auf eine Textur bringen. In diesem Fall soll allerdings eine **Sonnentextur** verwendet werden [1].



Implementierung Fragment-Shader Kaustik-Effekt

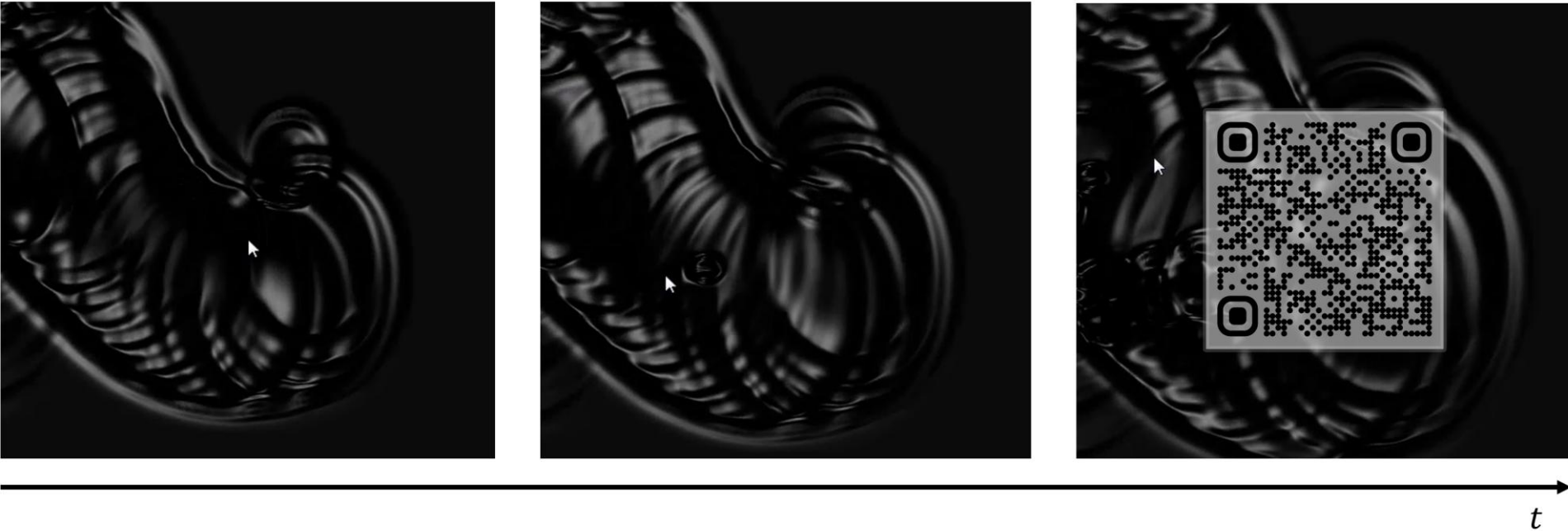
Schauen wir uns jetzt den Fragment-Shadercode für die **Kaustik** an:

```
...  
uniform sampler2D tex4; // Sonne  
  
void main(void) {  
    ...  
    vec4 kaustik = texture2D(tex4, 10.0*refractVector.xy + vec2(0.5, 0.5));  
    ...  
}
```



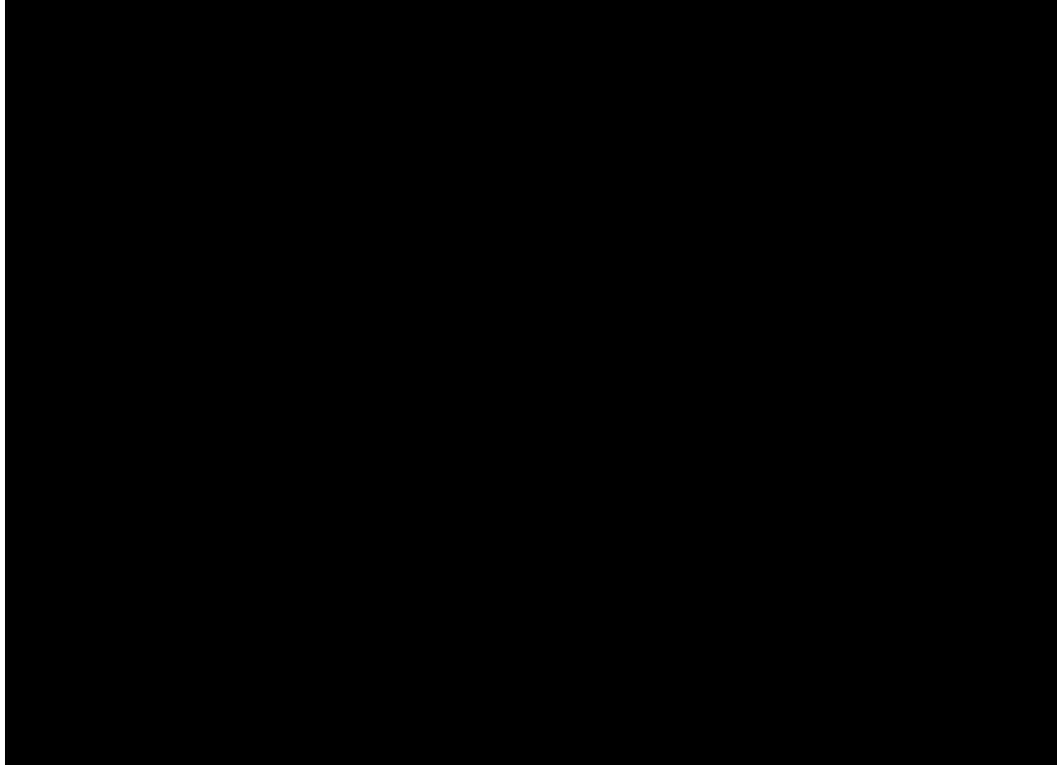
Kaustik-Effekt isoliert betrachtet

Hier sehen wir das isolierte Ergebnis des [Kaustik-Effekts](#) ohne die bereits bekannten Effekte, wie Refraktion und Schatten.



Implementierung im Shader

Schauen wir uns das Ergebnis der Kaustik an:



Programmcode (eingebettet):
kapitel02/InteraktiveWassersimulation.java

Refraktion, Schatten, Reflexion und Kaustik

Key Takeaways



- Refraktion beschreibt die Brechung von Licht beim Übergang zwischen zwei Medien
- Das Kreuzprodukt der Gradienten liefert die Oberflächennormale
- Das Gesetz von Snell beschreibt die Richtungsänderung gebrochener Lichtstrahlen
- Refraktion kann durch Verschiebung von Texturkoordinaten simuliert werden
- Einfache Schatteneffekte lassen sich direkt aus den Oberflächengradienten ableiten
- Reflexion basiert auf dem Prinzip: Einfallswinkel = Ausfallswinkel
- Kaustiken entstehen durch Bündelung und Aufweitung gebrochener Lichtstrahlen

Fragestellungen zum Vorlesungsteil

Im Anschluss an diese Veranstaltung sollten Sie die folgenden Fragen sicher beantworten können:

- (1) Beschreiben Sie, wie für die Refraktion aus der Höhenkarte des Wassers die Normale ermittelt werden kann.
- (2) Experimentieren Sie mit dem Gesetz von Snell und verwenden verschiedene Brechungsindizes.
- (3) Erläutern Sie an einem Beispiel, warum das Gesetz von Snell nicht genügt, um für eine Normale den Refraktionsvektor einfach zu bestimmen.
- (4) Ersetzen Sie die Methode **refract** durch die Formel aus der Vorlesung zur Bestimmung des Refraktionsvektors und testen Sie die Funktionalität.
- (5) Wie lässt sich der Reflexionsvektor herleiten?
- (6) Warum ist es nicht einfach möglich, beim Kaustik-Effekt, die Sonnenstrahlen durch die Wasseroberfläche zu schicken und dort zu akkumulieren? Dann sollte der Effekt doch sehr viel realistischer sein.

Praktische Aufgaben

Folgende praktische Aufgaben sollten Sie jetzt selbstständig durchführen:

Aufgabe 1) [Java] Wie ließe sich die Funktion **reflect** im Shadercode durch die Originalformel aus der Vorlesung ersetzen:

```
vec3 reflectVector    = reflect(n_view, n_surface);
```




Vielen Dank für die Aufmerksamkeit